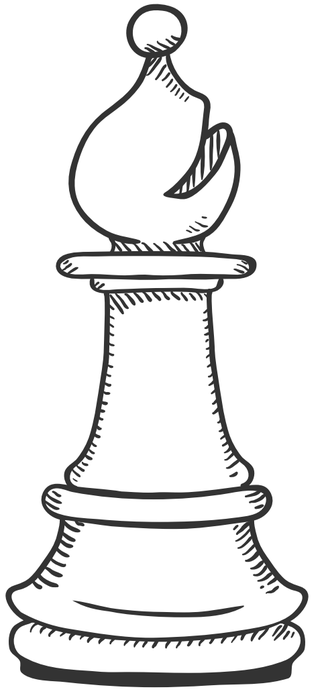
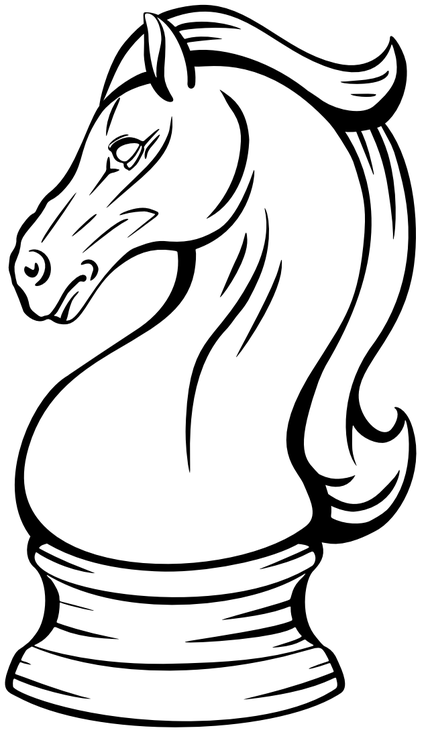
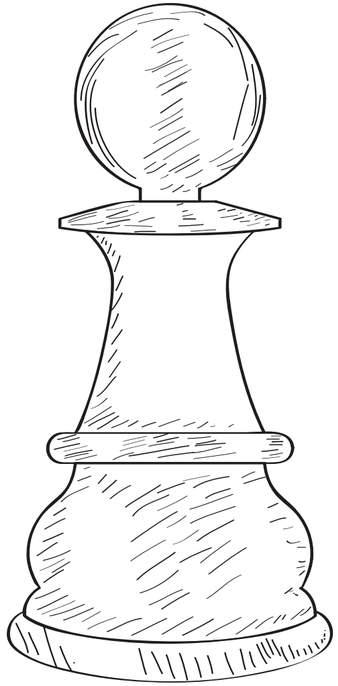
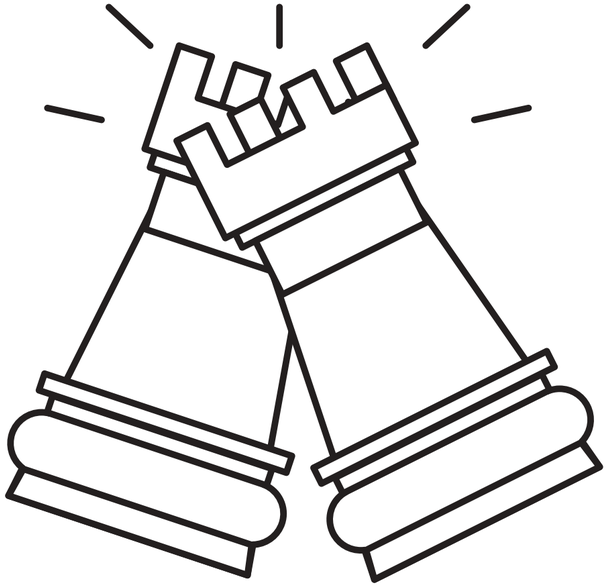
Latent Style Embeddings and Residual Weakness Modeling

Rating-Invariant Personalized Chess Training







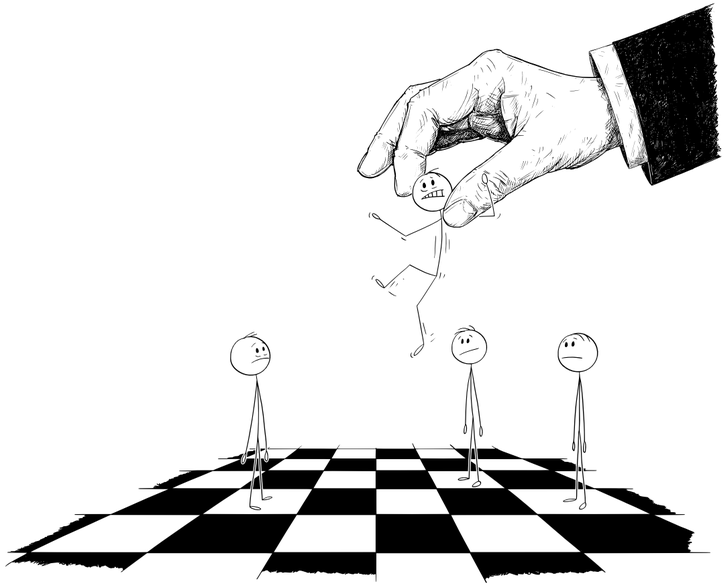


Problem

Current chess tools detect blunders but fail to model persistent, rating-adjusted weaknesses. We propose a transformer-based multi-task system that learns latent player style and residual error structure.

Existing analysis focuses on isolated tactical mistakes and does not separate strength from structural weaknesses, which means that two players with same rating may have very different weaknesses

Core Question



Can we disentangle player strength from structural decision-making tendencies by modeling residual errors relative to rating-conditioned expectations?

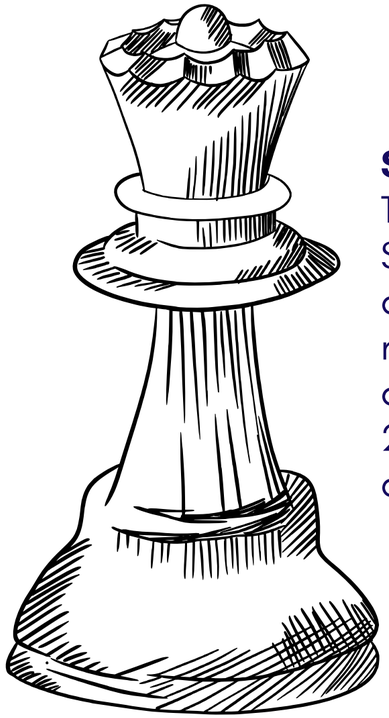
The Data (1)

```
[Event "F/S Return Match"]
[Site "Belgrade, Serbia JUG"]
[Date "1992.11.04"]
[Round "29"]
[White "Fischer, Robert J."]
[Black "Spassky, Boris V."]
[Result "1/2-1/2"]
```

The tags with the game data

```
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. O-O Be7 6. Re1 b5 7. Bb3 d6 8. c3
O-O 9. h3 Nb8 10. d4 Nbd7 11. c4 c6 12. cxb5 axb5 13. Nc3 Bb7 14. Bg5 b4 15.
Nb1 h6 16. Bh4 c5 17. dxe5 Nxe4 18. Bxe7 Qxe7 19. exd6 Qf6 20. Nbd2 Nxd6 21.
Nc4 Nxc4 22. Bxc4 Nb6 23. Ne5 Rae8 24. Bxf7+ Rxf7 25. Nxf7 Rxe1+ 26. Qxe1 Kxf7
27. Qe3 Qg5 28. Qxg5 hxg5 29. b3 Ke6 30. a3 Kd6 31. axb4 cxb4 32. Ra5 Nd5 33.
f3 Bc8 34. Kf2 Bf5 35. Ra7 g6 36. Ra6+ Kc5 37. Ke1 Nf4 38. g3 Nxh3 39. Kd2 Kb5
40. Rd6 Kc5 41. Ra6 Nf2 42. g4 Bd3 43. Re6 1/2-1/2
```

The movetext



Source

This project uses publicly available rated Standard chess games from the Lichess open database (*https://database.lichess.org*) . We restrict the dataset to Rapid and Classical time controls and players rated between 1200 and 2200 to reduce noise and maintain comparable decision-making contexts.

Structure

Each game is stored in PGN (Portable Game Notation) format and contains:

- Player metadata (ratings, time control, result)
- Sequential move data in UCI format
- Timestamps and game identifiers

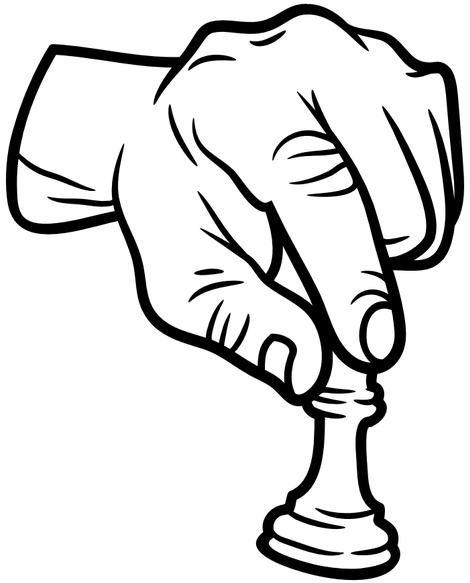
The data is inherently sequential and variable-length, requiring transformation into fixed-length training windows.

The Data (2)

Feature Representation
Each move is converted into structured model inputs including:

- From-square and to-square
- Promotion indicator
- Piece type and capture flags
- Game phase (opening, middlegame, endgame)
- Rating bucket

Sequences are windowed into fixed 128-ply segments to enable transformer-based training.



Data Treatment
To improve signal quality:

- Bullet and correspondence games are excluded
- Opening-book moves are filtered
- Extreme centipawn outliers are clipped
- Games are split chronologically for evaluation

The final dataset consists of structured, rating-conditioned sequential decision data suitable for multi-task representation learning.

Generated Labels
We compute centipawn loss for each move using Stockfish (chess engine) to obtain objective move quality. From this we derive:

- Expected move quality conditioned on rating
- Residual error (actual - expected)
- Phase-specific error aggregates

This allows separation of rating-consistent errors from structural weaknesses.

Model Architecture

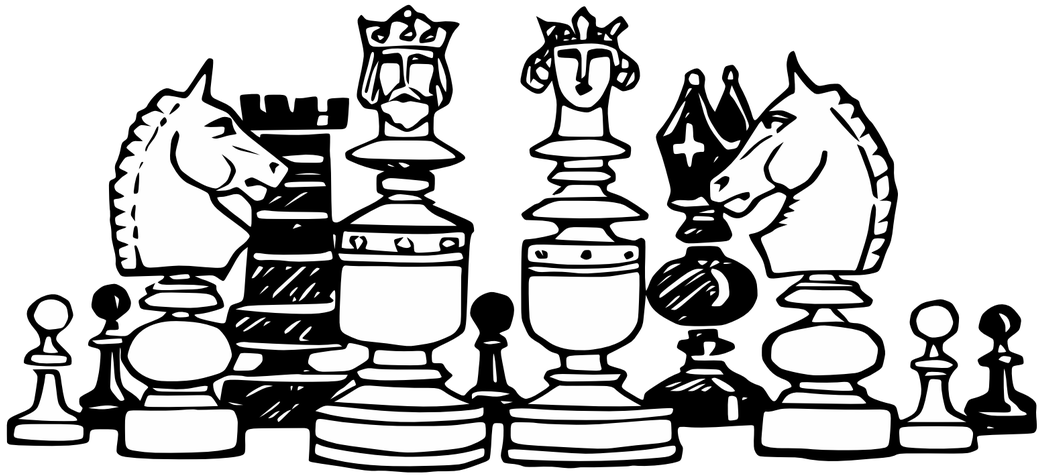
Overall Pipeline

PGN → Move Encoding → Transformer → Player Embedding → Multi-Task Heads

Raw games are converted into structured move tokens. A Transformer models the sequence of decisions and produces contextual representations. These are aggregated into a latent player embedding used for multiple learning objectives.

Transformer

- 4–6 layers
- 256 hidden dimension
- 8 attention heads



Heads

- Rating prediction
- Expected move quality
- Residual weakness modeling
- Contrastive style loss

Loss

$L = \lambda_1 L_rating$

- λ_2 L_expected
- λ_3 L_residual
- λ_4 L_contrastive

Key Innovation

Rating-Conditioned Residual Modeling
We model what should happen at a given rating, then measure how a player deviates from that expectation.

Residual = Actual move quality - Expected (given rating)

This separates:
- Rating-consistent mistakes
- Persistent structural weaknesses

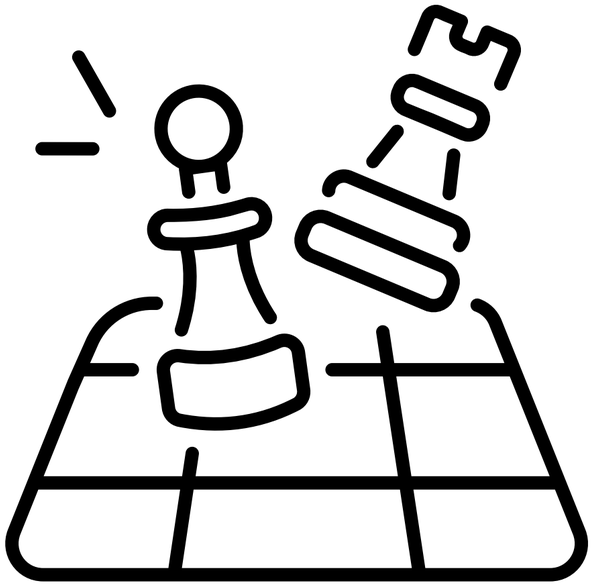
Two players with the same rating may have very different residual profiles.

Latent Style Embeddings
We use contrastive learning to bring games from the same player closer together in embedding space.

This produces:
- Player-level style vectors
- Clusterable decision patterns
- A visualizable representation of strategic tendencies

Why This Matters
Instead of just detecting blunders, the model identifies systematic patterns in decision-making. It moves from error detection to structural diagnosis.

Evaluation Plan



Representation Quality
We examine whether the learned embeddings meaningfully structure player behavior.

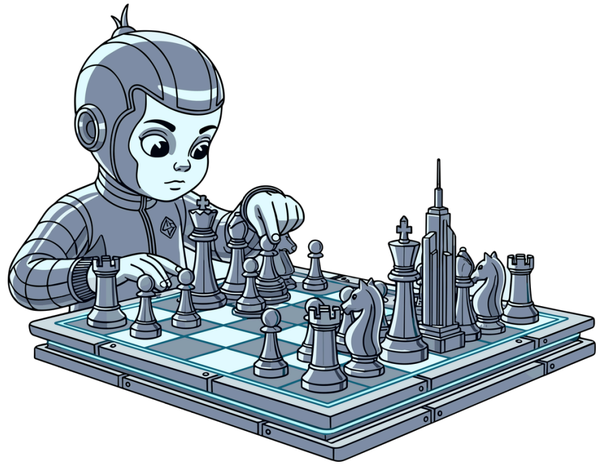
- UMAP visualization
- Cluster coherence analysis
- Separation across stylistic tendencies

Goal: verify that embeddings capture more than rating alone.

Residual Stability
We test whether weakness patterns are consistent over time.

- Chronological train-test split
- Within-player residual variance
- Phase-specific consistency

Goal: confirm residuals reflect persistent structure, not noise.



Personalization Validity
We evaluate whether predicted weaknesses generalize to future games.

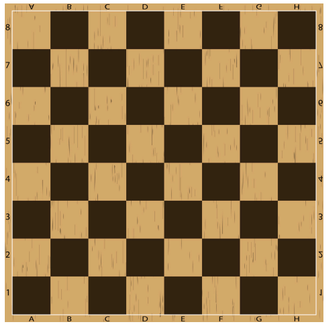
- Train on early games
- Predict weaknesses
- Validate against later performance

Goal: demonstrate real predictive utility.

Ethics, Privacy and Future Work

Ethical Considerations
This system is designed for diagnostic insight, not deterministic ranking. Residual error does not equal skill, and embeddings are not absolute measures of ability. Outputs are probabilistic and should support human interpretation rather than replace coaching. We avoid normative scoring and do not generate automated player rankings.

Data Privacy & Anonymity
We use publicly available Lichess data. Usernames are removed and replaced with hashed identifiers during preprocessing. No personally identifiable information is stored. Only rating, time control, and move sequences are retained. All processing is performed locally and no personal data is redistributed.



Bias & Fairness
Rating distribution and time-control differences may introduce bias.
We mitigate this by:

- Filtering to a defined rating band
- Restricting to Rapid and Classical games
- Modeling residuals relative to rating expectations

This reduces unfair comparisons across skill levels.

Future Work

- Scale to larger datasets
- Track style evolution over time
- Develop an interpretable training dashboard
- Deploy as an adaptive coaching API

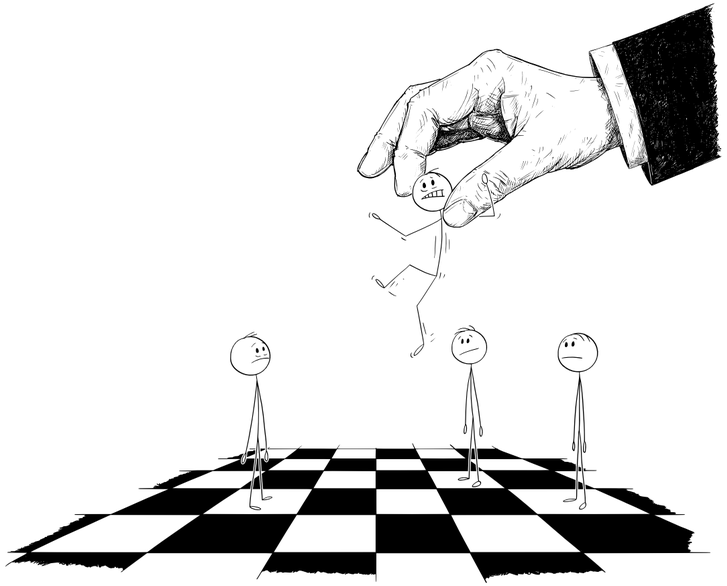
Related Work

McIlroy-Young et al., Maia: Human-Skill-Aware Chess Move Prediction (KDD 2020).
Aligning Superhuman AI with Human Behavior: Chess as a Model System
https://www.cs.toronto.edu/~ ashton/pubs/maia-kdd2020.pdf

McIlroy-Young et al., Behavioral Stylometry in Chess (2022).
Detecting Individual Decision-Making Style: Exploring Behavioral Stylometry in Chess
https://arxiv.org/abs/2208.01366

McIlroy-Young et al., Learning Models of Individual Behavior in Chess (KDD 2022).
Learning Models of Individual Behavior in Chess
https://www.cs.toronto.edu/~ ashton/pubs/maia-individual-kdd2022.pdf

My contribution



This project combines rating-aware modeling with residual error analysis to separate overall skill from consistent weaknesses in decision-making. Instead of only detecting isolated mistakes, we first estimate what performance should look like at a given rating and then measure how a player deviates from that expectation. By learning stable player embeddings and validating them over time, we move from basic blunder detection toward personalized, rating-adjusted diagnosis of structural weaknesses.

Q+A